QUE · LOCAL DEVELOPMENT

How to Run — Complete Local Setup Guide

Product: **Que** (SchemaGraph) · Path: `adc/schemagraph`
OS examples: **Windows PowerShell** (also works on macOS/Linux with small path tweaks)
Document version: **1.0** · Date: 29 Jul 2026
Goal: get UI + API + Postgres running on your machine for functional testing

This PDF lists **every component**, **every command**, env files, ports, demo logins, optional add-ons, and how to verify the stack is healthy.

Contents

1. What you will run

Process	What	Default URL / port	How started
1 Postgres	Metadata DB + pgvector (RAG)	`localhost:5432` · db `stitch`	Docker container `stitch-pg`
2 Que API	Express API (auth, sync, jobs, chat)	`http://localhost:8787`	`npm run dev` in `api/`
3 Que UI	Vite + React SPA	`http://localhost:5173`	`npm run dev` in `schemagraph/`

> You need **three terminals** for the recommended path (or Docker Compose for Postgres+API, plus Vite for UI). Warehouse connections (Snowflake / Databricks live) are optional — fixtures work offline.

```
Browser  →  Vite UI :5173  →  Que API :8787  →  Postgres :5432 (stitch)
                                    │
                                    └─ optional: live SF / DBX / PG customer_demo / Mongo
```

2. Prerequisites

Tool	Version / notes	Check command
Node.js	20+ recommended (API Docker uses 22)	`node -v`
npm	Comes with Node	`npm -v`
Docker Desktop	Running; for Postgres (+ optional Compose)	`docker version`
Git	Clone / pull repo	`git --version`
PowerShell or Terminal	Windows / macOS / Linux	—

> Postgres image must be `pgvector/pgvector:pg16` (not plain `postgres:16`), or AI RAG migration `007` fails.

3. Quick start (recommended path)

From a clean machine, run these in order. Details for each step follow in §§4–7.

```
# 0) Go to app root
cd D:\ADC\prosols\adc\schemagraph

# 1) Start / create Postgres (pgvector)
docker start stitch-pg
# If container does not exist yet — see Step A "First-time create"

# 2) API install + migrate + seed + start
cd api
npm install
npm run migrate
npm run seed
npm run dev
# Leave this terminal running → http://localhost:8787

# 3) UI install + start (new terminal)
cd D:\ADC\prosols\adc\schemagraph
npm install
npm run dev
# Leave this terminal running → open http://localhost:5173

# 4) Login (DEV build shows demo shortcuts)
#    Owner:  dev@stitch.local / stitch-dev
```

> **Success =** `GET http://localhost:8787/health` returns `{"ok":true, … }` and the UI login page loads; after login you see the Workspace canvas.

4. Step A — Postgres (pgvector)

A1. If container already exists

```
docker start stitch-pg
docker ps --filter name=stitch-pg
```

> Status should show `Up` and ports `0.0.0.0:5432→5432/tcp`.

A2. First-time create

```
docker run -d --name stitch-pg `
  -e POSTGRES_USER=stitch `
  -e POSTGRES_PASSWORD=stitch `
  -e POSTGRES_DB=stitch `
  -p 5432:5432 `
  pgvector/pgvector:pg16
```

macOS / Linux (same, without backticks — one line or `\`):

```
docker run -d --name stitch-pg \
  -e POSTGRES_USER=stitch -e POSTGRES_PASSWORD=stitch \
  -e POSTGRES_DB=stitch -p 5432:5432 \
  pgvector/pgvector:pg16
```

A3. Connection details

Setting	Value
Host	`localhost`
Port	`5432`
Database	`stitch`
User	`stitch`
Password	`stitch`
URL	`postgresql://stitch:stitch@localhost:5432/stitch`

```
# Optional: open psql inside container
docker exec -it stitch-pg psql -U stitch -d stitch
```

> If an old `stitch-pg` was created without pgvector, remove and recreate:
> `docker stop stitch-pg; docker rm stitch-pg` then run A2 again, then remigrate + reseed.

5. Step B — Migrations

Preferred: ordered migrator

```
cd D:\ADC\prosols\adc\schemagraph\api
npm install
npm run migrate
```

Applies `db/001_*.sql` … `db/014_*.sql` in order (skips already applied; tracks `schema_migrations`).

Includes: auth, jobs, RAG/pgvector, secrets, notebooks, join HITL, attestation audit, OIDC state + invites.

> Console ends with `[Que] migrations up to date` or lists newly applied files as `ok`.

> Customer demo SQL (`002_customer_demo.sql` , `002b_…`) is **not** auto-applied by the migrator (filtered out). Use bootstrap scripts in Step E if you need the live PG demo source.

6. Step C — API env, install, start

C1. Create / edit `api/.env`

File path: `adc/schemagraph/api/.env` (loaded automatically at API boot).

```
# Minimum local
DATABASE_URL=postgresql://stitch:stitch@localhost:5432/stitch
PORT=8787
DEMO_WORKSPACE_ID=22222222-2222-2222-2222-222222222222

# Recommended local (stable encryption + attestation)
QUE_SECRETS_KEY=que-local-dev-secrets-key-change-me-32c
QUE_ATTESTATION_HMAC_SECRET=que-local-attestation-hmac-dev

# Keep auth ON for realistic testing
STITCH_AUTH_DISABLED=false

# Demo users (owner/member/viewer) — OK for local only
QUE_SEED_DEMO_USERS=true

# CORS — optional locally (defaults include localhost:5173)
# QUE_CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173

# Do NOT set QUE_ENV=production for day-to-day local work
# (production boot requires real secrets + CORS and blocks auth bypass)
```

C2. Install & start API

```
cd D:\ADC\prosols\adc\schemagraph\api
npm install
npm run migrate
npm run seed
npm run dev
```

`npm run dev` = `node --watch src/index.js` (auto-restarts on file change).

Production-style (no watch): `npm start`.

> Look for: `que-api listening on http://localhost:8787`
> Demo seed may log that owner/member/viewer passwords were ensured.

C3. API scripts reference

Command	Purpose
`npm run dev`	API with file watch
`npm start`	API once
`npm run migrate`	Apply SQL migrations
`npm run seed`	Seed demo workspace / fixtures metadata
`npm run bootstrap:demo-source`	Postgres `customer_demo` source
`npm run bootstrap:demo-mongo`	Mongo demo collections
`npm run bootstrap:test-data`	Extra test data
`npm run test:diligence`	Joins + privacy + functional tests
`npm run test:smoke`	Smoke vs running API

7. Step D — Frontend env, install, start

D1. Create / edit `.env.local` at SPA root

File path: `adc/schemagraph/.env.local`

```
VITE_STITCH_API_URL=http://localhost:8787
VITE_STITCH_LIVE_ONLY=1
```

Variable	Meaning
`VITE_STITCH_API_URL`	API base URL the browser calls
`VITE_STITCH_LIVE_ONLY=1`	If API is down, show empty — not dummy offline schema

D2. Install & start UI

```
cd D:\ADC\prosols\adc\schemagraph
npm install
npm run dev
```

> Vite prints a Local URL — usually `http://localhost:5173`. Open it in the browser.

Other UI scripts: `npm run build` · `npm run preview` · `npm run lint` · typecheck: `npx tsc --noEmit -p tsconfig.app.json`

8. Step E — Seed & demo sources (optional)

After API is running at least once (or from `api/` with DB up):

```
cd D:\ADC\prosols\adc\schemagraph\api

# Core metadata / workspace seed
npm run seed

# Optional: live Postgres customer_demo database + connection
npm run bootstrap:demo-source

# Optional: Mongo demo
docker start stitch-mongo
# First time:
# docker run -d --name stitch-mongo -p 27017:27017 mongo:7
npm run bootstrap:demo-mongo
```

> On **Sources** you can also add **Databricks / Snowflake fixture** connections (no cloud account) and click Sync — enough for join + job testing.

9. Daily startup (after first setup)

```
# Terminal 1 — Postgres
docker start stitch-pg

# Terminal 2 — API
cd D:\ADC\prosols\adc\schemagraph\api
npm run dev

# Terminal 3 — UI
cd D:\ADC\prosols\adc\schemagraph
npm run dev
```

Only re-run `npm run migrate` after pulling new `db/*.sql` files. Re-seed only if you wiped the DB.

10. Docker Compose alternative

Starts **Postgres + API** together (UI still on host Vite).

```
cd D:\ADC\prosols\adc\schemagraph
docker compose up --build
```

- Applies migrations inside the API container, then listens on `:8787`.
- Postgres service name in Compose network: `stitch-pg`.
- Then start UI: `npm run dev` in `schemagraph/` with `VITE_STITCH_API_URL=http://localhost:8787`.

```
# Stop Compose stack
docker compose down

# Stop and delete DB volume (full reset)
docker compose down -v
```

11. Login, URLs & smoke checks

URLs

What	URL
UI login	`http://localhost:5173/login`
Workspace	`http://localhost:5173/workspace`
Sources / Jobs / Chat / Settings	`/sources` · `/jobs` · `/chat` · `/settings`
API health	`http://localhost:8787/health`
OpenAPI stub	`http://localhost:8787/openapi.json`

Demo accounts (local DEV only)

Role	Email	Password
Owner	`dev@stitch.local`	`stitch-dev`
Member	`member@stitch.local`	`stitch-member`
Viewer	`viewer@stitch.local`	`stitch-viewer`

> Demo shortcuts appear on the login page only in Vite **development** builds. Production UI leaves fields blank.

Smoke commands

```
# Health
curl http://localhost:8787/health

# Login (PowerShell)
$body = @{ email = "dev@stitch.local"; password = "stitch-dev" } | ConvertTo-Json
Invoke-RestMethod -Method POST -Uri http://localhost:8787/auth/login `
  -ContentType "application/json" -Body $body

# Expect: ok=true, token, workspaces[]
```

> **Correct health shape:** `{"ok":true,"service":"que-api","authDisabled":false,"sso":"not_configured"|"ready"}`

12. Optional: LLM, GitHub, SSO, Mongo

LLM (AI chat)

Not required — skills/heuristics work without keys. For LLM mode, either:

- UI → **Settings** → **BYOK** (workspace encrypted keys), or
- Add to `api/.env`:

```
OPENAI_API_KEY=sk- ...
# and/or
ANTHROPIC_API_KEY=sk-ant- ...
```

Then enable **Prefer LLM for AI chat** in Settings. Reindex: Settings or `POST ... /ai/reindex`.

GitHub dbt PR export

In Settings: set `githubOwner`, `githubRepo`, branch, dbt path; save **GitHub token** via secrets UI.
Or env: `GITHUB_TOKEN=ghp_ ...`

SSO (OIDC) — local IdP

```
# api/.env additions
QUE_OIDC_ISSUER=https://your-idp.example.com
QUE_OIDC_CLIENT_ID= ...
QUE_OIDC_CLIENT_SECRET= ...
QUE_OIDC_REDIRECT_URI=http://localhost:8787/auth/sso/callback
QUE_OIDC_POST_LOGIN_REDIRECT=http://localhost:5173/auth/callback
QUE_SSO_ALLOWED_DOMAINS=yourcompany.com
QUE_SSO_REQUIRE_INVITE=true
```

Register the API callback URL in the IdP. Token returns in URL **hash** (`#token=`), never query.

Mongo demo

```
docker run -d --name stitch-mongo -p 27017:27017 mongo:7
cd D:\ADC\prosols\adc\schemagraph\api
npm run bootstrap:demo-mongo
```

13. Automation tests

```
cd D:\ADC\prosols\adc\schemagraph\api

# No live server required for diligence suite
npm run test:diligence
# = eval:joins + test:privacy + test:functional

# Optional: API must already be running
npm run test:smoke

# Frontend typecheck
cd D:\ADC\prosols\adc\schemagraph
npx tsc --noEmit -p tsconfig.app.json
```

Manual checklist: `docs/MANUAL_TEST_PLAN.md` · Feature oracles: `docs/Que-Feature-Guide.pdf`

14. Troubleshooting

Symptom	Likely cause	Fix
API crash: connection refused :5432	Postgres not up	`docker start stitch-pg`
Migration 007 fails (vector)	Wrong Postgres image	Recreate with `pgvector/pgvector:pg16`
UI loads but no data / auth errors	Wrong API URL or API down	Check `.env.local`; open `/health`
CORS errors in browser console	Origin not allowlisted	Set `QUE_CORS_ORIGINS` to include `http://localhost:5173`
Login 401 with demo user	Seed skipped / wrong password	`QUE_SEED_DEMO_USERS=true`, restart API, or `npm run seed`
Port 8787 / 5173 / 5432 in use	Another process	Stop other app or change `PORT` / Vite port
API exits mentioning production secrets	`QUE_ENV=production` or `NODE_ENV=production`	Unset for local; or set real keys + CORS
Docker Desktop not running	Engine stopped	Start Docker Desktop, wait until ready

15. Stop / reset

```
# Stop processes
# Ctrl+C in API and UI terminals

docker stop stitch-pg
# optional: docker stop stitch-mongo

# Full DB wipe (destroys all local Que metadata)
docker stop stitch-pg
docker rm stitch-pg
# then recreate with Step A2 → migrate → seed
```

16. Command cheat sheet

```
# ═══ FIRST TIME ═══
cd D:\ADC\prosols\adc\schemagraph
docker run -d --name stitch-pg -e POSTGRES_USER=stitch -e POSTGRES_PASSWORD=stitch -e
POSTGRES_DB=stitch -p 5432:5432 pgvector/pgvector:pg16

cd api
npm install
npm run migrate
npm run seed
npm run dev

# new terminal
cd D:\ADC\prosols\adc\schemagraph
npm install
# ensure .env.local has VITE_STITCH_API_URL=http://localhost:8787
npm run dev

# ═══ EVERY DAY ═══
docker start stitch-pg
cd api; npm run dev
# other terminal: cd schemagraph; npm run dev

# ═══ VERIFY ═══
curl http://localhost:8787/health
# Browser: http://localhost:5173/login
# Login: dev@stitch.local / stitch-dev

# ═══ TESTS ═══
cd api; npm run test:diligence

# ═══ COMPOSE (API+DB) ═══
cd schemagraph; docker compose up --build
```

Folder map

Path	Role
`adc/schemagraph/`	SPA (Vite) root
`adc/schemagraph/.env.local`	Frontend env
`adc/schemagraph/api/`	Express API
`adc/schemagraph/api/.env`	API env
`adc/schemagraph/db/`	SQL migrations
`adc/schemagraph/docker-compose.yml`	Postgres + API containers
`adc/schemagraph/docs/`	This guide + feature/manual docs

Que Local Setup Guide v1.0 · Companion PDFs: `Que-Feature-Guide.pdf`, checklist `MANUAL_TEST_PLAN.md`.
Repo path examples use `D:\ADC\prosols\adc\schemagraph` — adjust if your clone lives elsewhere.